Bahria University, Islamabad

Department of Software Engineering

Artificial Intelligence Lab

(Spring-2026)

Teacher: Engr Saad Mazhar

Student : Umama Hidayat Khan

Enrollment : 01-131232-107

Lab Journal: 5

Date: March 3, 2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

LAB

Lab 5: A* Algorithm

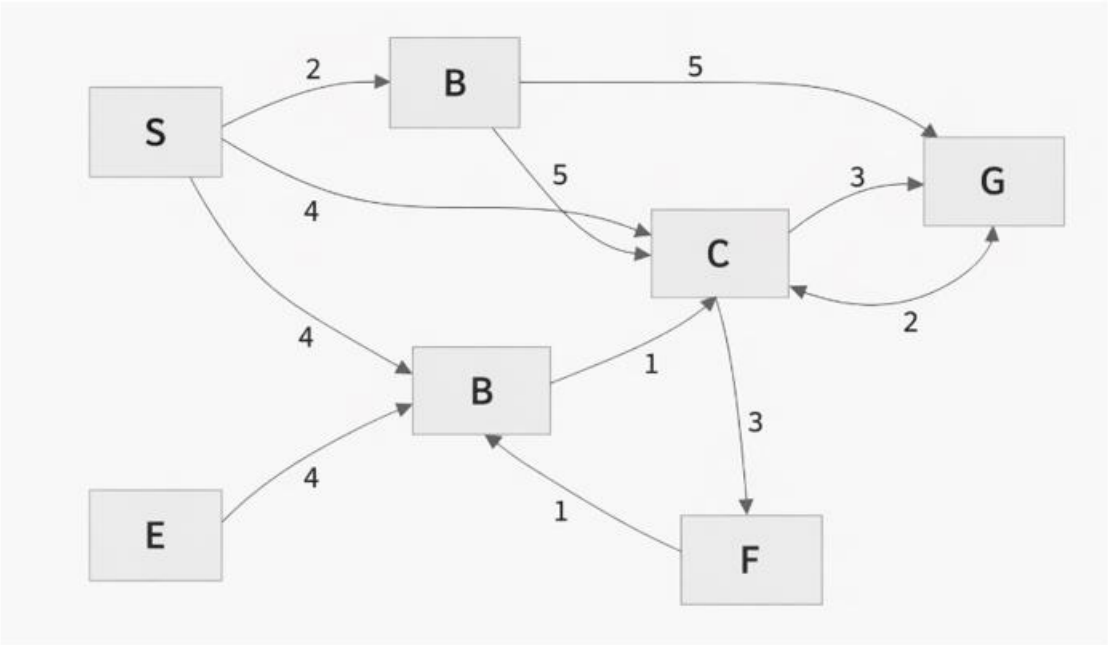
Introduction:

Search algorithms are used to explore graphs and find paths between nodes. DFS, BFS, and A* are three commonly used search techniques, each with a different strategy for exploring nodes. They vary in terms of data structure used, efficiency, optimality, and practical applications.

Tools Used:

VS Code

Graph:



Task 1:

(1) represents this graph in code (adjacency list)

Code:

```
# Graph Representation (Adjacency List)

graph = {
    'S': [('B1', 4), ('B2', 2), ('C', 4)],
    'B1': [('C', 1)],
    'B2': [('C', 5), ('G', 5)],
    'C': [('G', 3), ('F', 3), ('G', 2)],
    'E': [('B1', 4)],
    'F': [('B1', 1)],
    'G': [('C', 2)]
}
```

Task 2:

(2) implements DFS and BFS to find any valid path from S to G by treating every edge as unweighted (ignore weights for DFS/BFS)

Code:

```
# DFS

def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()
    expanded = 0

    while stack:
        node, path = stack.pop()

        if node not in visited:
            visited.add(node)
            expanded += 1

            if node == goal:
                return path, expanded

            for neighbor, _ in graph[node]:
                stack.append((neighbor, path + [neighbor]))

    return None, expanded
```

```
# BFS

def bfs(graph, start, goal):
    queue = deque([(start, [start])])
    visited = set()
    expanded = 0

    while queue:
        node, path = queue.popleft()

        if node not in visited:
            visited.add(node)
            expanded += 1

            if node == goal:
                return path, expanded

            for neighbor, _ in graph[node]:
                queue.append((neighbor, path + [neighbor]))

    return None, expanded
```

Task 3:

(3) implements A* to find the minimum-cost path from S to G using the given edge weights and a heuristic function you define and justify (e.g., h(n)=0 to reduce to Dijkstra, or a simple admissible estimate based on node layout)

Code:

```
# A* Algorithm

def heuristic(node):

    # Using h(n) = 0 reduces A* to Dijkstra
    return 0


def a_star(graph, start, goal):
    priority_queue = []
    heapq.heappush(priority_queue, (0, start, [start], 0))   # (f, node, path,
g)


    visited = set()
    expanded = 0

    while priority_queue:
        f, node, path, g = heapq.heappop(priority_queue)
```

```
        if node not in visited:
            visited.add(node)
            expanded += 1

            if node == goal:
                return path, expanded, g

            for neighbor, cost in graph[node]:
                new_g = g + cost
                new_f = new_g + heuristic(neighbor)
                heapq.heappush(priority_queue, (new_f, neighbor, path +
[neighbor], new_g))

    return None, expanded, None
```

Task 4:

(4) prints for each algorithm the final path and the number of expanded nodes (and for A* also print the total path cost)

Code:

```
if __name__ == "__main__":

    start_node = 'S'
    goal_node = 'G'

    print("===== DFS =====")
    dfs_path, dfs_expanded = dfs(graph, start_node, goal_node)
    print("Path:", dfs_path)
    print("Expanded Nodes:", dfs_expanded)

    print("\n===== BFS =====")
    bfs_path, bfs_expanded = bfs(graph, start_node, goal_node)
    print("Path:", bfs_path)
    print("Expanded Nodes:", bfs_expanded)

    print("\n===== A* =====")
    astar_path, astar_expanded, astar_cost = a_star(graph, start_node, goal_node)
    print("Path:", astar_path)
    print("Expanded Nodes:", astar_expanded)
    print("Total Cost:", astar_cost)
```

Output:

```
===== DFS =====
Path: ['S', 'C', 'G']
Expanded Nodes: 3
```

```
===== BFS =====
Path: ['S', 'B2', 'G']
Expanded Nodes: 5
```

```
===== A* =====
Path: ['S', 'C', 'G']
Expanded Nodes: 5
Total Cost: 6
```

Task 5:

(5) writes a short comparison that clearly differentiates DFS vs BFS vs A* in terms of data structure used (stack/queue/priority queue), completeness, optimality, time complexity, space complexity, and when each should be used in real applications.

DFS, BFS, and A* are search algorithms used to find paths in a graph, but they differ in strategy and efficiency.

DFS uses a **stack (LIFO)** and explores as deep as possible before backtracking. It is **not optimal** and may not be complete in infinite graphs because it can follow a long path before checking shorter ones. Its time complexity is **O(V + E)** and space complexity is **O(V)**. DFS is useful when any solution is acceptable and memory usage needs to be lower.

BFS uses a **queue (FIFO)** and explores nodes level by level. It is **complete and optimal for unweighted graphs**, since it guarantees the shortest path in terms of number of edges. Its time complexity is **O(V + E)**, but it may use more memory than DFS. BFS is commonly used when the shortest path is required in unweighted scenarios.

A* uses a **priority queue** and selects nodes based on the minimum value of f(n)=g(n)+h(n)f(n) = g(n) + h(n)f(n)=g(n)+h(n). It is **complete and optimal** if the heuristic is admissible. Its time complexity is approximately **O(E log V)** due to priority queue operations. A* is widely used in real-world applications like pathfinding, GPS navigation, and robotics because it efficiently finds the lowest-cost path.

In short, DFS explores deeply, BFS explores broadly, and A* explores intelligently using cost and heuristics.

Conclusion:

In conclusion, DFS, BFS, and A* differ mainly in how they explore the graph and guarantee solutions. DFS focuses on depth, BFS focuses on level-by-level exploration, and A* focuses on finding the most cost-efficient path using heuristics. The choice of algorithm depends on whether we prioritize speed, memory usage, or optimal path cost.